

AI Graph Engineering · Module 1.2 Revision Note

Neo4j & Cypher basics — turning your drawing into a queryable graph

Phase 1 (Foundations) | Prepared for Babatunde | 24 Aug 2026 | Mentor session 2

Cypher is your drawing, typed out. Nodes go in round brackets (), relationships in square brackets [], and the arrow --> is literally the arrow you drew. You sketch the shape you want and the database finds every place that shape exists. That's the whole language — "ASCII-art pattern matching."

1 · The pattern skeleton — read every part

```
( t : Test { name: 'LoginTest' } ) - [ :CALLS ] -> ( s : Service )  
| | | | | |  
| LABEL KEY VALUE REL TYPE | LABEL  
variable (alias, so you can RETURN t) node/alias
```

The () as a whole = a **node**. The [] = a **relationship**. The alias (t, s) lets you refer to it later in WHERE / RETURN.

2 · The colon rule (your #1 fix)

Colon = TYPE. Dot = DATA. A colon introduces a label or a relationship type; a dot reaches into a property.

- (s:Service) = a node of type Service ✓ (s.Service) = WRONG (dot is for properties)
- [:REQUIRES] = a relationship of type REQUIRES ✓ [REQUIRES] = WRONG (no colon = matches EVERY edge type)
- s.name = the name property of s ✓ (dot is correct here — it's data)

3 · The core clauses

Clause	Does	Example
CREATE	Write new nodes/relationships (always inserts).	CREATE (:Test {name:'LoginTest'})
MATCH	Find an existing pattern (the mirror of CREATE).	MATCH (t:Test)-[:CALLS]->(s:Service)
WHERE	Filter matched rows.	WHERE s.name = 'PaymentService'
RETURN	Choose what to output.	RETURN t.name
MERGE	Match if it exists, else create (idempotent).	MERGE (:Service {name:'LoginService'})

4 · CREATE vs MERGE = the idempotency idea

CREATE is "always insert" — run your script twice and you get duplicate nodes. **MERGE** is "insert only if missing" — run it a thousand times, you get exactly one. This is **idempotency**, and you already live it: a good fixture / setUp() can re-run without creating a second test user. MERGE is CREATE with that safety — which is why graph-building scripts (Module 1.3) use MERGE so a service referenced by 12 tests becomes **one** node, not twelve.

5 · Silent wrong-answer traps (these don't error — they mislead)

- **Missing colon on a relationship:** [CALLS] is read as a variable and matches EVERY edge type. Use [:CALLS].
- **Label case-sensitivity:** :TEST ≠ :Test. A mismatched label returns zero rows, not an error. Match labels exactly as created.
- **Unpinned node:** (auth) matches ANY node. Pin it: (auth:Service {name:'LoginService'}) or filter in WHERE.

6 · Worked queries on your e-commerce model

```
// build it idempotently  
MERGE (auth:Service {name:'LoginService'})  
MERGE (pay:Service {name:'PaymentService'})
```

```

MERGE (:Test {name:'CheckoutTest'})-[:CALLS]->(pay)
MERGE (pay)-[:REQUIRES]->(auth)

// blast radius: services that require Auth
MATCH (s:Service)-[:REQUIRES]->(Service {name:'LoginService'})
RETURN s.name

// aggregation: how many services require Auth
MATCH (s:Service)-[:REQUIRES]->(Service {name:'LoginService'})
RETURN count(s)

// TWO-HOP: tests whose service depends on Auth
MATCH (t:Test)-[:CALLS]->(Service)-[:REQUIRES]->(Service {name:'LoginService'})
RETURN DISTINCT t.name

```

7 · Teach-it-back one-liners

“Cypher is your drawing typed out: () is a node, [] is a relationship, --> is the arrow.”

“Colon = type, dot = data. (:Label) and [:TYPE] both need the colon; s.name uses the dot.”

“Forget the colon on a relationship and Cypher matches EVERY edge — a silent wrong answer.”

“Neo4j labels are case-sensitive: :Test and :TEST are different; a mismatch returns nothing.”

“CREATE always inserts; MERGE inserts only if missing. MERGE is a re-runnable fixture.”

8 · Your assignment & status

Module 1.2 stays OPEN until Assignment 1.2 is in — the gap is syntax fluency, and reps against the live database are the cure. **Do:** re-load the model with MERGE (prove idempotency by running twice), then write 5 queries incl. one aggregation and one two-hop; save the working ones to qa-graph-lab/cypher/module-1.2.cypher. When a query misbehaves, read the error/empty result and fix it yourself — that debugging IS the learning. **Next:** Module 1.3 — Python that parses a real repo and builds the graph for you (MERGE-based).